

HEART DISEASE PREDICTION

Internship Report submitted in partial fulfillment of the
requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

By

Muddam Sai Manish Yadav
21R11A05D5



Department of Computer Science and Engineering
Accredited by NBA

Geethanjali College of Engineering and Technology
(UGC Autonomous)

(Affiliated to J.N.T.U.H, Approved by AICTE, New Delhi)
Cheeryal (V), Keesara (M), Medchal. Dist.-501301.

September-2023

Geethanjali College of Engineering and Technology

(UGC Autonomous)

(Affiliated to JNTUH, Approved by AICTE, New Delhi)

Cheeryal (V), Keesara (M), Medchal Dist.-501301.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Accredited by NBA



CERTIFICATE

This is to certify that the Internship Report entitled “**Heart Disease Prediction**” is a bonafide work done by **Muddam Sai Manish Yadav(21R11A05D5)** in partial fulfillment of the requirements of the award for the degree of Bachelor of Technology in “**Computer Science and Engineering**” from Jawaharlal Nehru Technological University, Hyderabad during the year 2023-2024.

HOD-CSE

Dr. A. Sree Lakshmi

Professor

Examiner

Signature:

Name:

Designation:

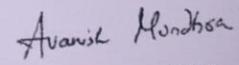
CERTIFICATE

OF PROTERNSHIP

THIS CERTIFICATE IS PRESENTED TO

M. Sai Manish Yadav

On successful completion of a Proternship program at
Cantilever Labs from 15 May, 2023 to 21 July, 2023



AVANISH MUNDHRA
FOUNDER & CEO
CANTILEVER LABS

Geethanjali College of Engineering and Technology

(UGC Autonomous)

(Affiliated to JNTUH Approved by AICTE, New Delhi)

Cheeryal (V),Keesara (M),MedchalDist. -501301.

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Accredited by NBA



DECLARATION BY THE CANDIDATE

I, **Muddam Sai Manish Yadav**, bearing Roll No. **21R11A05D5**., hereby declare that the Internship Report entitled “**Heart Disease Prediction**” is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering**. This is a record of bonafide work carried out by me in **Cantilever** and the results embodied in this Internship report have not been reproduced or copied from any source. The results embodied in this Internship Report have not been submitted to any other University or Institute for the award of any other degree or diploma.

Muddam Sai Manish Yadav,

21R11A05D5,

Department of CSE,

Geethanjali College of Engineering and Technology,

Cheeryal.

ACKNOWLEDGEMENT

During my internship in the field of Artificial Intelligence and Machine Learning at Cantilever, I have had the privilege to work alongside a highly talented and supportive team. The guidance and mentorship I received throughout this program have been invaluable in enhancing my skills and knowledge in AI and ML. I am sincerely grateful for the trust placed in me and for the meaningful projects and tasks I was given, which allowed me to gain hands-on experience in AI model development and data analysis and machine learning. I am extremely thankful for the rewarding internship experience at Cantilever, and I look forward to continuing my journey in the exciting field of Artificial Intelligence and Machine Learning with the skills and knowledge I have acquired during my time here.

Muddam Sai Manish Yadav

21R11A05D5

Introduction about Internship Organization

Cantilever Labs is driven by the passion to build a win-win platform that assists students to secure their dream jobs and support corporates to find the right candidates. We pave the road to your success.

A roadmap just for you: We provide the best of experiential learning ambience and pre-hand experience with practical exercises for complete preparation, with 100% assurance for placements in any desired company. We work towards building a confident personality that would crack any entrance interviews, alongside with intellectual aptitude skills, quantitative skills, Verbal and soft-skills, Profile-building with radical thinking to crack any question with varied level of difficulty. With extensible hours of training and proficient methods of teaching, technical training, mock interviews with HRs of top companies like Google, Microsoft, Amazon etc. The candidate will be prepped, brushed and full-fledged equipped for any interview in a product-based company or competitive environment.

Personalisation: Cantilever Lab's focus on making our teaching as personalised as possible based on the client analysis of requirements. Personalised lesson plans, schedules to learn at your own pace to land the internship that you desire. Technical skills, aptitude skills, competitive programming, now personalised, just for you. To equip your desired talents with a 100% field-proficiency. With a wide range of opportunities and options to choose from on this roller coaster to your corporate success.

We are Customer Centric

The primary focus is the client and their goals, we work on the overall personality development of the candidate. Whilst focusing on technical development and Verbal skills based on the client's level of interest, we imbibe core values and confidence development through the technical courses designed for you, that will aid in building the path to a placement in companies like Google, Microsoft, Amazon, etc.

Industry Exposure: With the help of experienced faculties from IIT's and IIM's, our linchpin is the efficient course material and study patterns for all courses, with aiding the candidate in networking through exposure in practical fields and projects in real-time. Mock-interviews and internships at top companies help in creating an impactful presence of the candidate in the assigned position in a wide environment of contacts. Landing internships and jobs in highly reputed brands and companies

through our placement preparation, just a click away!

Lab's expertise: At Cantilever Labs, we build a candidate from the programming skills to the final rounds of interview, inclusive of Mock GD's, one-on-one interviews, resume making kits etc. We maintain a concrete experience of learning and building a professional experience for everyone.

Training Schedule

Induction Session: 15-05-2023

Training start Date: 16-05-2023

Project Training: 21-06-2023 to 01-07-2023

Project submission Duration: 03-07-2023 to 24-07-2023

TABLE OF CONTENTS

S.No	Contents	Page No
	Abstract	i
	List of Figures	ii
	List of Screenshots	iii
	List of Symbols & Abbreviations	iv
1	Introduction	1
2	Related work/Technologies used	3
3	Work done/Observation/Duties performed	6
4	Learning after Internship	22
5	Summary/Conclusion	23
6	Bibliography	24

ABSTRACT

Heart disease is a broad term for many different heart conditions. This can include coronary heart disease, a heart attack, or heart failure. Heart disease is a prevalent and life-threatening condition that affects millions of people worldwide.

Early and accurate detection of heart diseases is crucial for effective treatment and prevention of potential cardiac events. The Heart Disease Prediction System is a data-driven project designed to aid healthcare professionals in predicting the likelihood of heart disease in patients based on various risk factors.

The Heart Disease Prediction System demonstrates the power of machine learning in healthcare and its role in early detection and prevention of heart diseases. By providing timely and accurate predictions, the system holds promise in reducing the burden of heart-related conditions and improving overall public health. The project encourages further research and development in predictive analytics for improved patient care and well-being.

These are completely implemented natively in python. Here we use Kaggle data sets for implementing the prediction system. These are implemented using python tools such as Anaconda and Jupyter note book and used the following libraries such as numpy, pandas, sklearn, etc.

LIST OF FIGURES

S.No	Figure No	Title of Figure	Page No
1	2.1.1	Machine Learning	3

LIST OF SCREENSHOTS

S.No	Figure No	Title Of Figure	Page No
1	3.2.1	Importing Libraries and Loading Data	7
2	3.2.2	Handling Missing Values and Duplicate Data	8
3	3.2.3	Data Processing & Encoding Categorical Data	9
4	3.2.4	Feature Scaling	10
5	3.2.5	Splitting the Dataset into Training and Train Sets	11
6	3.2.6.1	Logistic Regression	12
7	3.2.6.2	Importing svm library	12
8	3.2.6.3	Using kNeighbors classifier	14
9	3.2.6.4	Decision tree classifier	14
10	3.2.6.5	Random forest classifier	15
11	3.2.6.6	Gradient boosting classifier	16
12	3.2.7	Choosing best model	17
13	3.2.8	Prediction on New Data	18
14	3.2.9	Saving the Best Model	19

10	3.2.10	GUI	21
----	--------	-----	----

LIST OF ABBREVIATION

S.No	Abbreviation	Full Form
1	GUI	Graphical User Interface
2	SVM	Support vector machine
3	AI&ML	Artificial Intelligence &Machine Learning

1.INTRODUCTION

1.1 Heart Disease

Heart diseases, encompassing a wide range of cardiovascular conditions, have emerged as a significant global health concern. These conditions affect the heart and blood vessels, posing serious threats to individuals' well-being and leading to a substantial number of fatalities worldwide. Factors such as sedentary lifestyles, poor dietary habits, smoking, and various medical conditions contribute to the prevalence of heart diseases. In response to this growing health challenge, there is a pressing need for early detection and risk assessment systems to identify high-risk individuals and provide timely interventions.

1.2 Heart Disease Prediction

The Heart Disease Prediction System is a sophisticated and innovative healthcare solution designed to forecast the risk of developing heart diseases in individuals. It utilizes the capabilities of machine learning, and data analysis to assess the probability of heart-related conditions based on various risk factors and medical data. The primary objective of this system is to empower healthcare professionals and individuals with early detection tools, enabling timely interventions and personalized preventive measures.

1.3 Benefits of Heart Disease Prediction

The Heart Disease Prediction System marks a significant advancement in healthcare technology, revolutionizing the approach to heart disease management.

- **Early Detection:** By identifying individuals at high risk of heart diseases at an early stage, the system facilitates timely interventions and preventive measures, potentially preventing severe health complications.
- **Resource Optimization:** Healthcare providers can optimize their resources by focusing on high-risk patients and providing them with the necessary attention and care.

- Research and Development: The vast amount of data collected and analyzed by the system can contribute to ongoing research on heart diseases, leading to advancements in medical knowledge and treatment options.
- Population Health Improvement: The system's ability to identify population-level heart disease trends enables public health initiatives to address broader health concerns.

2.RELATED WORK/TECHNOLOGIES USED

2.1 Machine Learning

Machine learning is a method of data analysis that automates analytical model building. It is a branch of artificial intelligence based on the idea that systems can learn from data, identify patterns and make decisions with minimal human intervention.

Machine Learning, as the name says, is all about machines learning automatically without being explicitly programmed or learning without any direct human intervention. This machine learning process starts with feeding them good quality data and then training the machines by building various machine learning models using the data and different algorithms. The choice of algorithms depends on what type of data we have and what kind of task we are trying to automate.

As for the formal definition of Machine Learning, we can say that a Machine Learning algorithm learns from **experience E** with respect to some type of **task T** and **performance measure P**, if its performance at tasks in T, as measured by P, improves with experience E.

For example, if a Machine Learning algorithm is used to play chess. Then the experience E is playing many games of chess, the task T is playing chess with many players, and the performance measure P is the probability that the algorithm will win in the game of chess. The Fig-2.1.1 is a picture showing about machine learning.



Fig-2.1.1 Machine learning

2.2 Data Analysis and Preprocessing

Data analysis and preprocessing techniques are crucial to ensure the quality and relevance of the data used in the prediction system. Data cleaning, feature selection, normalization, and handling missing values are some of the essential data preprocessing steps undertaken to prepare the data for training machine learning models.

2.3 Python

Python is a popular programming language in data science and machine learning due to its versatility and a wide range of libraries and frameworks. Python is used to implement machine learning models, perform data manipulation.

2.3.1 Pandas and Numpy

Pandas and NumPy are Python libraries extensively used for data manipulation, data analysis, and numerical computations. Pandas provide powerful data structures like Data Frames and Series, while NumPy enables efficient numerical operations on arrays.

2.3.2 Scikit-learn

Scikit-learn is a widely-used machine learning library in Python that offers a diverse set of tools for classification, regression, clustering, and more. It provides implementations of various machine learning algorithms, making it convenient for building predictive models.

2.3.3 Matplotlib and Seaborn

These libraries are used for data visualization and creating informative graphs and plots. They aid in visually interpreting the data and model performance, enabling better insights and decision-making.

2.3.4 Standard Scaler

Standard Scaler from scikit-learn is employed for feature scaling, ensuring that all numerical features have a mean of zero and a standard deviation of one. This step is crucial for ensuring that different features are on similar scales and do not bias the machine learning models.

2.3.5 Tkinter

Tkinter is a standard Python library used for creating graphical user interfaces (GUIs). It is utilized to design and implement the user interface of the Heart Disease Prediction System, allowing users to input their health data and receive risk assessments.

2.3.6 Joblib

Joblib is used to save and load machine learning models. It helps persist the trained models, allowing them to be reused for predictions without retraining.

3.WORK DONE

3.1 How It Works?

- The project starts with the importation of essential libraries and the loading of the heart disease dataset ('heart.csv'). The dataset is carefully examined to address missing values and duplicate records, ensuring data integrity and reliability.
- Data processing involves the segregation of categorical and continuous variables. Categorical features are one-hot encoded to facilitate their inclusion in the machine learning models. Meanwhile, continuous variables undergo feature scaling using Standard Scaler, enabling a level playing field for accurate model training.
- The dataset is then split into training and testing sets to facilitate model development and evaluation. Several machine learning algorithms, including Logistic Regression, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), Decision Tree, Random Forest, and Gradient Boosting, are employed to predict heart disease with high accuracy.
- The models are thoroughly evaluated, and their performance metrics, particularly accuracy scores, are compared. The model exhibiting the highest accuracy is selected as the most suitable predictor for heart disease.
- The project culminates in the creation of a user-friendly Graphical User Interface (GUI). Users can input their relevant health parameters, such as age, sex, blood pressure, cholesterol levels, and other vital signs, to obtain personalized heart disease risk predictions. The trained model powers real-time predictions, empowering individuals to make informed decisions about their heart health.

3.2 Implementation Procedure

3.2.1 Importing Libraries and Loading Data

In this step, we import necessary Python libraries like Pandas for data manipulation, NumPy for numerical computations, scikit-learn that contains wide range of algorithms and tools for data processing and model evaluation, Matplotlib for data visualization, Seaborn that provides high-level

interface for creating attractive statistical graphics , Tkinter for creating interactive graphical user interfaces and Joblib for saving and loading machine learning models. We also load the heart disease dataset using Pandas. Fig 3.2.1 shows the imported libraries and data set.

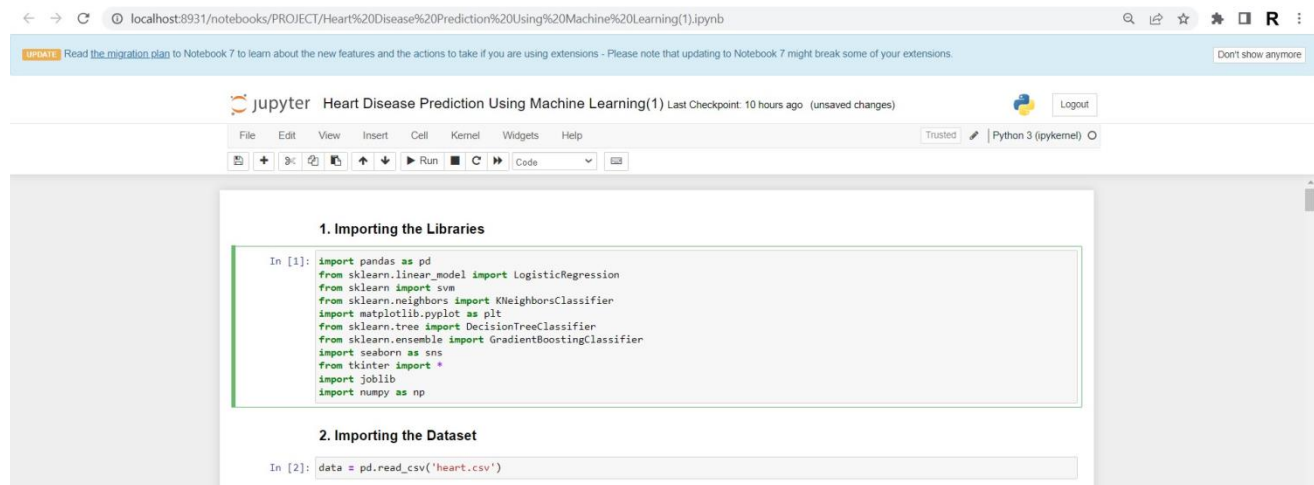


Fig 3.2.1 Importing Libraries and Loading Data

3.2.2 Handling Missing Values and Duplicate Data

In this step we perform data cleaning by checking for missing values and duplicate data in the heart disease dataset. The `null().sum()` function helps us identify the number of missing values in each column, the output indicates that there are no missing values in any column. The `duplicated().any()` function is used to check for duplicate rows, if duplicate rows exist, we remove them from the dataset using the `drop_duplicates()` function. The fig 3.2.2 is about handling missing values and duplicate data.

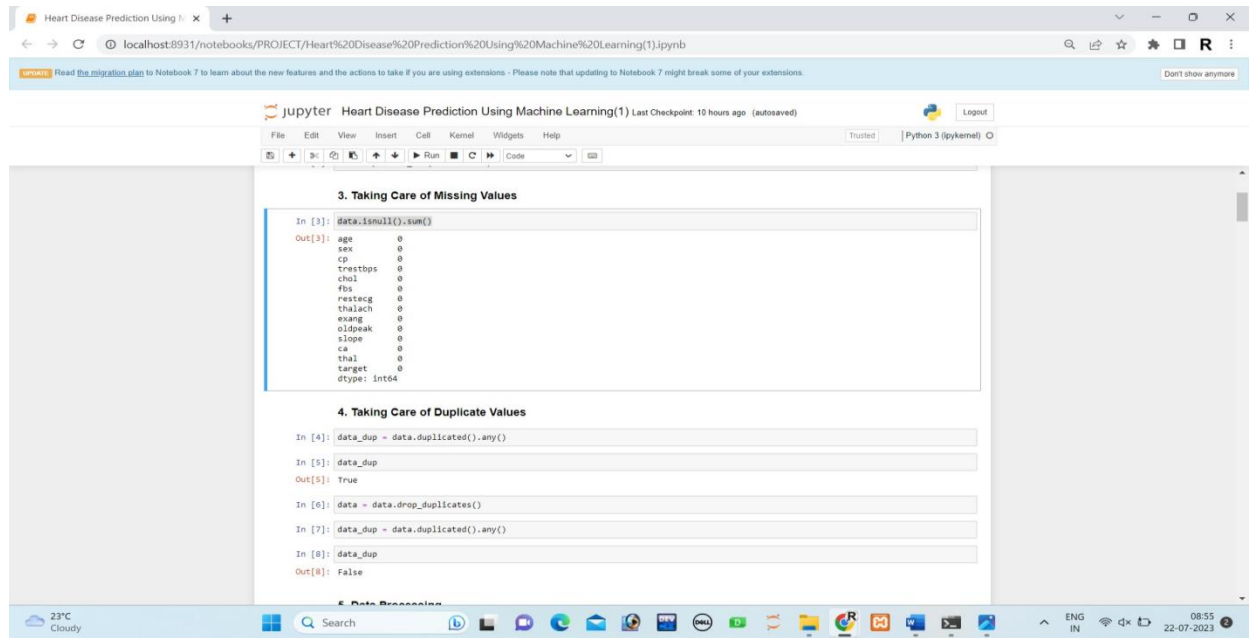


Fig 3.2.2 Handling Missing Values and Duplicate Data

3.2.3 Data Processing& Encoding Categorical Data

In this step, we prepare the data for machine learning. We categorize columns as categorical or continuous based on the number of unique values they contain. Then, we encode the categorical features using one-hot encoding to make them suitable for machine learning algorithms. The fig 3.2.3 includes data processing and encoding categorical data.

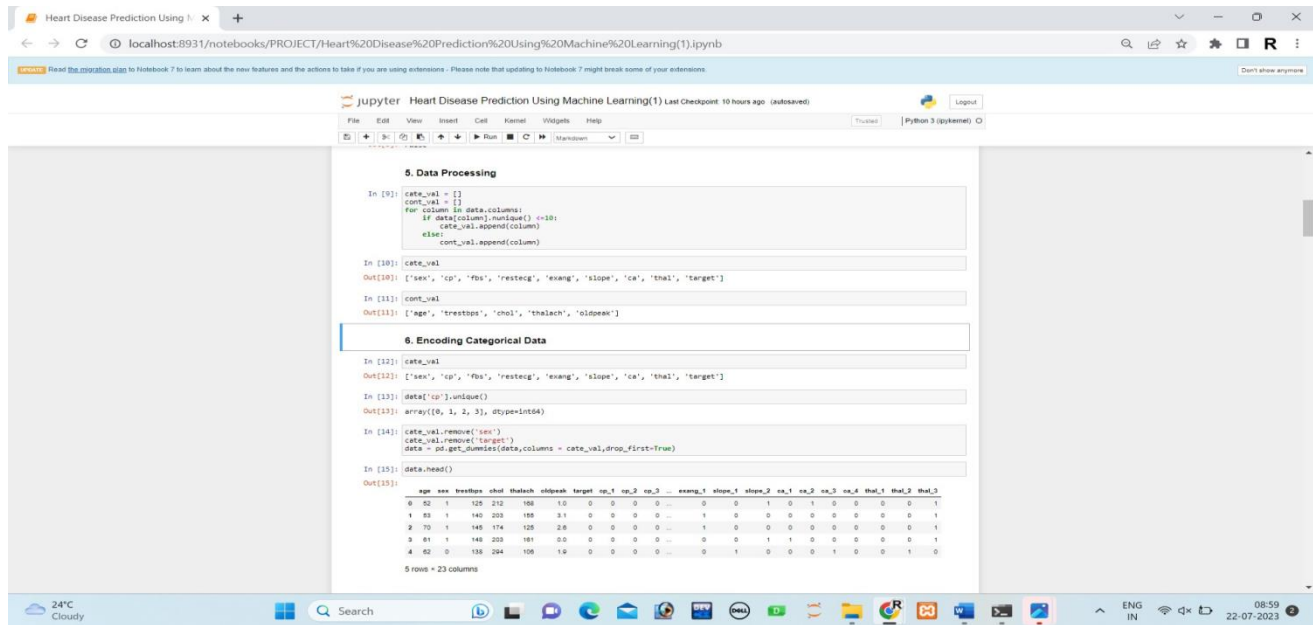


Fig 3.2.3 Data Processing & Encoding Categorical Data

3.2.4 Feature Scaling

Feature scaling is essential to ensure that all features are on a similar scale, avoiding any bias in the machine learning models. We use Standard Scaler from scikit-learn to scale the continuous features. The fig 3.2.4 includes feature scaling. Here we import the Standard Scaler library

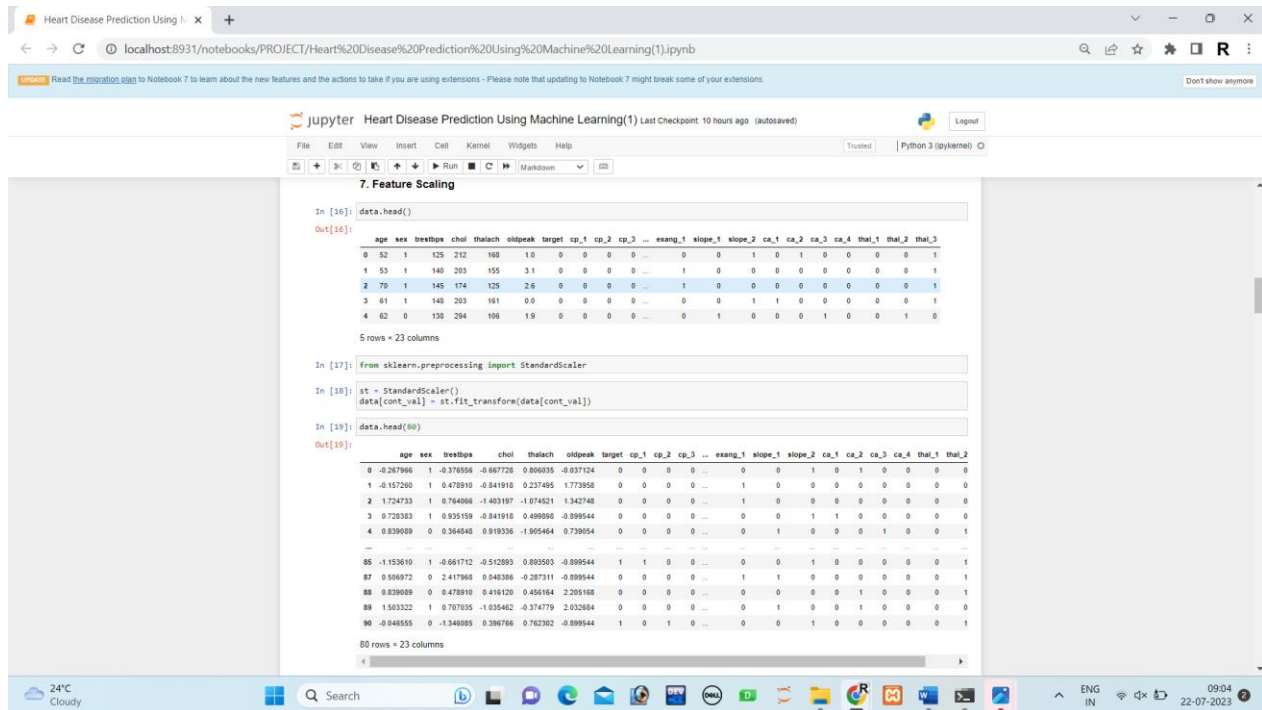
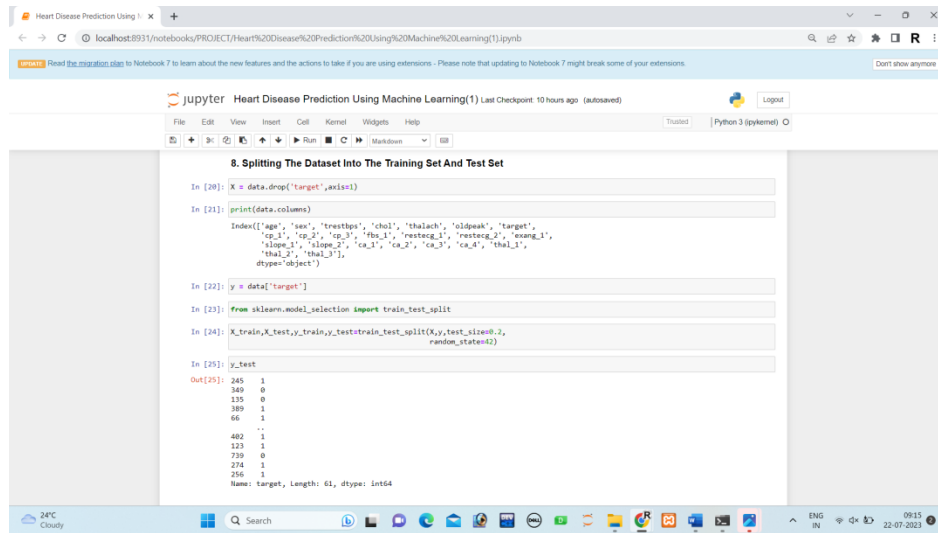


Fig 3.2.4 Feature Scaling

3.2.5 Splitting the Dataset into Training and Test Sets

To evaluate the performance of our machine learning models, we split the dataset into training and test sets. The training set is used to train the models, while the test set is used to evaluate their accuracy of the model.

In this code, X represents the feature variables (all columns except the 'target' column) and Y represents the target variable (the 'target' column). The fig 3.2.5 snippets the splitting the data set into Training and Test sets.



```
8. Splitting The Dataset Into The Training Set And Test Set

In [20]: X = data.drop("target",axis=1)

In [21]: print(data.columns)
Index(['age', 'sex', 'trestbps', 'chol', 'thalach', 'oldpeak', 'target',
       'ca_1', 'ca_2', 'ca_3', 'ca_4', 'restecg', 'restng_d', 'exang_d',
       'slope_1', 'slope_2', 'ca_1', 'ca_2', 'ca_3', 'ca_4', 'thal_1',
       'thal_2', 'thal_3'],
      dtype='object')

In [22]: y = data["target"]

In [23]: from sklearn.model_selection import train_test_split

In [24]: X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,
      random_state=42)

In [25]: y_test
Out[25]: 245    1
         349    0
         135    0
         389    1
           66    1
           ..
         482    1
         123    1
         729    0
         274    1
         256    1
         Name: target, Length: 61, dtype: int64
```

Fig 3.2.5 Splitting the Dataset into Training and Train Sets

3.2.6 Training and evaluating models

We train several machine learning models, including Logistic Regression, SVM, KNeighbors Classifier, Decision Tree Classifier, Random Forest Classifier, and Gradient Boosting Classifier, on the training data. After training, we use the test data to make predictions and calculate the accuracy of each model.

3.2.6.1 LOGISTIC REGRESSION

In this step, we train a logistic regression model using the training data and evaluate its performance on the test data. The accuracy of the logistic regression model is approximately 78.69%. The fig 3.2.6.1 shows using the function logistic regression.

The screenshot shows a Jupyter Notebook titled "Heart Disease Prediction Using Machine Learning(1)". The notebook is running on a local host. The code in the notebook is as follows:

```

In [26]: data.head()
Out[26]:
  age  sex  trestbps  chol  trestbps  oldpeak  target  cp  1  cp  2  cp  3  ...  exang  1  slope  1  slope  2  ca  1  ca  2  ca  3  ca  4  thal  1  thal  2  thal  3
126796 1  0.370506 -0.667728 0.806030 -0.037124 0  0  0  0  0  0  ...  0  0  1  0  1  0  0  0  0  0  1
1187560 1  0.478010 -0.841918 0.339480 1.970006 0  0  0  0  0  0  ...  1  0  0  0  0  0  0  0  0  0  1
1724752 1  0.764056 1.403197 -1.074521 1.342748 0  0  0  0  0  0  ...  1  0  0  0  0  0  0  0  0  0  1
1728383 1  0.535105 -0.841918 0.495858 -0.899544 0  0  0  0  0  0  ...  0  0  1  1  0  0  0  0  0  0  1
1829059 0  0.364840 0.919336 -1.905404 0.739004 0  0  0  0  0  0  ...  0  1  0  0  0  0  1  0  0  0  1  0

```

The code continues with importing the LogisticRegression model from sklearn, training the model, and evaluating its performance using accuracy_score. The final output shows an accuracy of 0.7868852459816393.

Fig.3.2.6.1 Logistic Regression

3.2.6.2 SVM

In this step, we train an SVM model using the training data and evaluate its performance on the test data. The accuracy of the SVM model is approximately 80.33%. This fig 3.2.6.2 includes importing of svm library.

The screenshot shows a Jupyter Notebook titled "Heart Disease Prediction Using Machine Learning(1)". The code in the notebook is as follows:

```

In [30]: from sklearn.metrics import accuracy_score
In [31]: accuracy_score(y_test, y_pred1)
Out[31]: 0.7868852459816393

10. SVC
In [32]: from sklearn import svm
In [33]: svm = svm.SVC()
In [34]: svm.fit(X_train, y_train)
Out[34]: SVC

In [35]: y_pred2 = svm.predict(X_test)
In [36]: accuracy_score(y_test, y_pred2)
Out[36]: 0.8032786885245982

11. KNeighbors Classifier
In [37]: from sklearn.neighbors import KNeighborsClassifier
In [38]: knn = KNeighborsClassifier()

```

Fig 3.2.6.2 Importing svm library

3.2.6.3 KNeighbors Classifier

In this step, we train a KNeighbors Classifier model using the training data and evaluate its performance on the test data. We find the optimal value of k (number of neighbors) by plotting the accuracy for different k values. The accuracy is approximately 73.77%.

Heart Disease Prediction Using 1. x

localhost:8931/notebooks/PROJECT/Heart%20Disease%20Prediction%20Using%20Machine%20Learning(1).ipynb

UPDATE Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions. Don't show anymore

jupyter

Heart Disease Prediction Using Machine Learning(1) Last Checkpoint: 11 hours ago (autosaved)

Logout

File Edit View Insert Cell Kernel Widgets Help

Trusted Python 3 (pykernel)

11. KNeighbors Classifier

```
In [37]: from sklearn.neighbors import KNeighborsClassifier
In [38]: knn = KNeighborsClassifier()
In [39]: knn.fit(X_train,y_train)
Out[39]: <KNeighborsClassifier:
KNeighborsClassifier()

In [40]: y_pred3=knn.predict(X_test)
In [41]: accuracy_score(y_test,y_pred3)
Out[41]: 0.7377049180327869

In [42]: score = []
for k in range(1,40):
    knn=KNeighborsClassifier(n_neighbors=k)
    knn.fit(X_train,y_train)
    y_pred=knn.predict(X_test)
    score.append(accuracy_score(y_test,y_pred))

In [43]: score
Out[43]: [0.7213114754098361,
0.8032786885245902,
0.7049180327868853,
0.7049180327868853,
0.7377049180327869,
0.8032786885245902,
```

The screenshot shows a web browser window displaying a Jupyter Notebook. The browser's address bar shows the URL: `localhost:8931/notebooks/PROJECT/Heart%20Disease%20Prediction%20Using%20Machine%20Learning(1).ipynb`. A notification bar at the top of the notebook interface states: "UPDATE! Read the migration plan to Notebook 7 to learn about the new features and the actions to take if you are using extensions - Please note that updating to Notebook 7 might break some of your extensions." The notebook title is "Heart Disease Prediction Using Machine Learning(1)" and it indicates the last checkpoint was 11 hours ago (autosaved). The interface includes a menu bar (File, Edit, View, Insert, Cell, Kernel, Widgets, Help), a toolbar with icons for file operations and execution, and a "Python 3 (ipykernel)" environment selector. The main content area shows a code cell with the following text:

```
In [43]: score
Out[43]: [0.7213114754098361,
0.8032786885245902,
0.7049180327868853,
0.7049180327868853,
0.7377049180327869,
0.8032786885245902,
0.7868852459016393,
0.8032786885245902,
0.7704918032786885,
0.7540983606557377,
0.7704918032786885,
0.7704918032786885,
0.7377049180327869,
0.7377049180327869,
0.7540983606557377,
0.7704918032786885,
0.7540983606557377,
0.7540983606557377,
0.7377049180327869,
0.7540983606557377,
0.7377049180327869,
0.7213114754098361,
0.7377049180327869,
0.7377049180327869,
0.7213114754098361,
0.7377049180327869,
0.7377049180327869,
0.7377049180327869,
0.7377049180327869,
0.7377049180327869,
0.7377049180327869,
0.7377049180327869,
0.7377049180327869,
0.7377049180327869,
0.7377049180327869]
```

The bottom of the image shows a Windows taskbar with various application icons and a system tray displaying the date and time as 22-07-2023, 09:29.

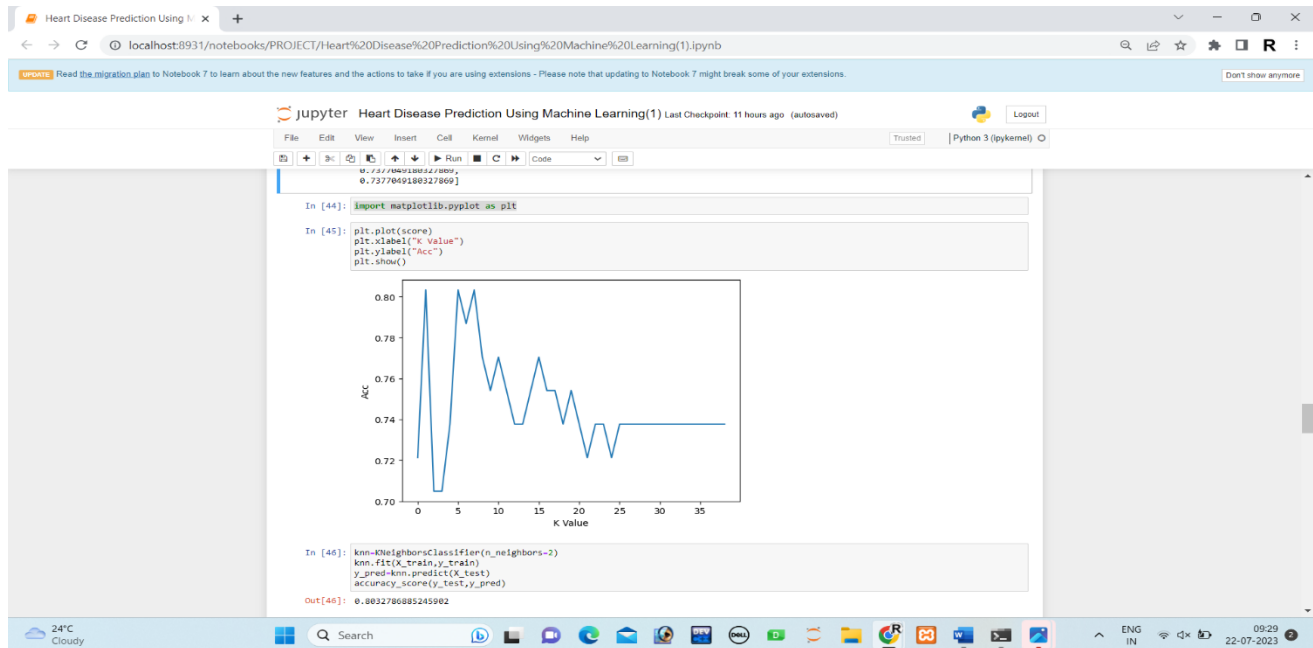


Fig 3.2.6.3 Using KNeighbors Classifier

3.2.6.4 Decision Tree Classifier

In this step, we train a Decision Tree Classifier model using the training data and evaluate its performance on the test data. The accuracy of the Decision Tree Classifier is approximately 73.77%. Fig 3.2.6.4 includes importing DecisionTreeClassifier library.

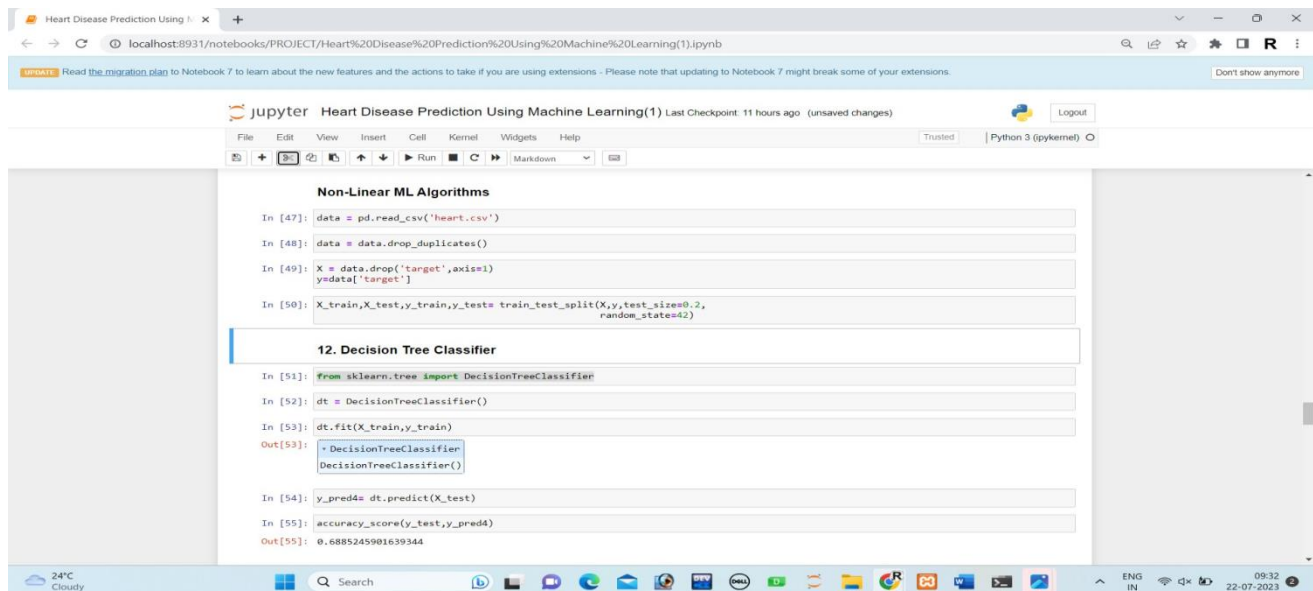
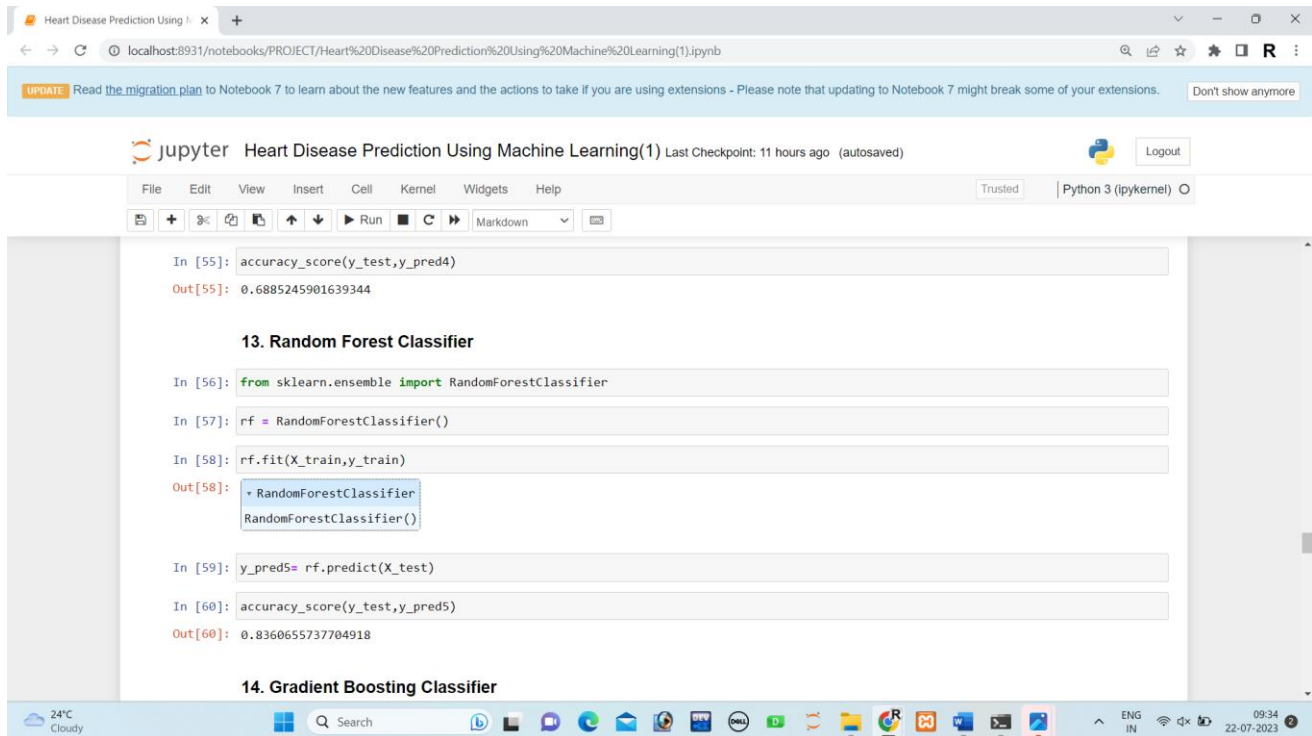


Fig 3.2.6.4 Decision tree classifier

3.2.6.5 Random Forest Classifier

In this step, we train a Random Forest Classifier model using the training data and evaluate its performance on the test data. The accuracy of the Random Forest Classifier is approximately 83.61%. The fig 3.2.6.5 is about random forest classifier showing accuracy of the project.



```
In [55]: accuracy_score(y_test,y_pred4)
Out[55]: 0.6885245901639344

13. Random Forest Classifier

In [56]: from sklearn.ensemble import RandomForestClassifier
In [57]: rf = RandomForestClassifier()
In [58]: rf.fit(X_train,y_train)
Out[58]: RandomForestClassifier()

In [59]: y_pred5= rf.predict(X_test)
In [60]: accuracy_score(y_test,y_pred5)
Out[60]: 0.8360655737704918

14. Gradient Boosting Classifier
```

Fig 3.2.6.5 Random Forest Classifier

3.2.6.6 Gradient Boosting Classifier

In this step, we train a Gradient Boosting Classifier model using the training data and evaluate its performance on the test data. The accuracy of the Gradient Boosting Classifier is approximately 80.33%. Fig 3.2.6.6 shows gradient boosting classifier showing its accuracy.

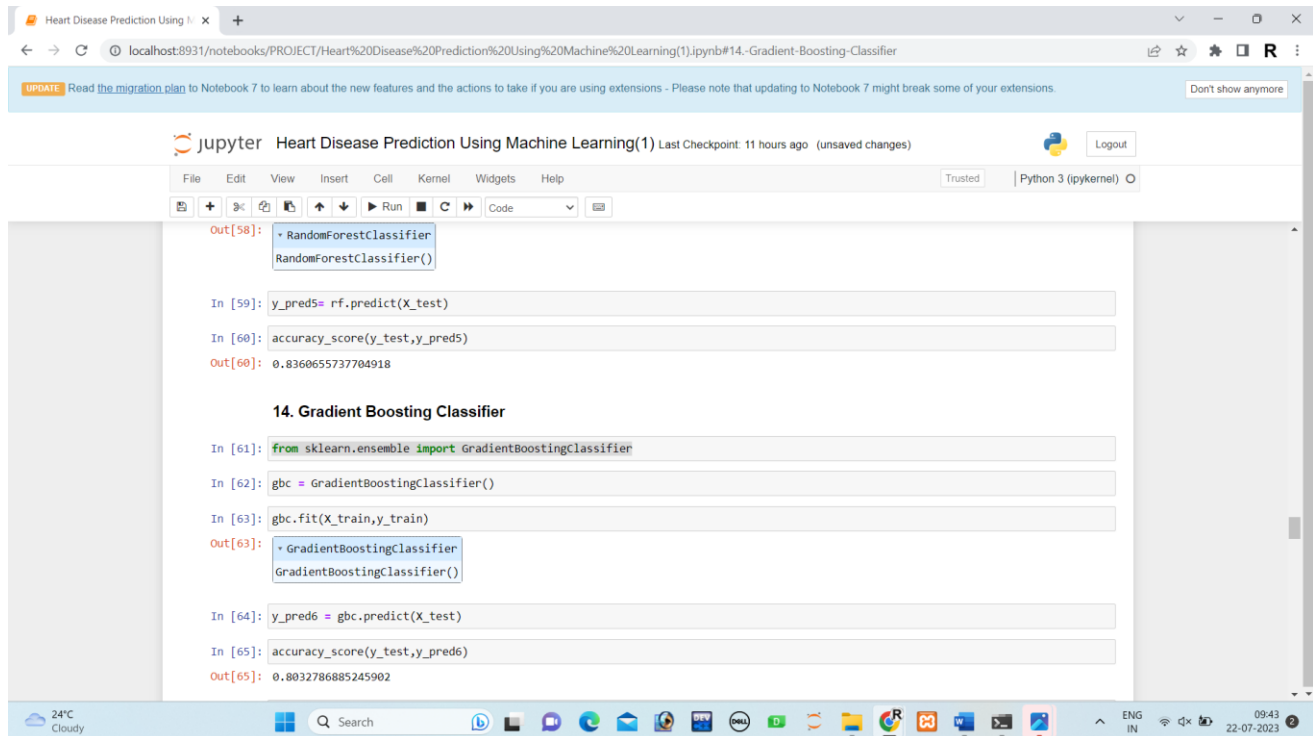


Fig 3.2.6.6 Gradient boosting classifier

3.2.7 Comparison of Models and selecting the best model

In this step, we compare the accuracy of all the models and select the one with the highest accuracy as our best model for the Heart Disease Prediction System.

Among all the models, the Random Forest Classifier achieved the highest accuracy of approximately 83.61%. Therefore, we select the Random Forest Classifier as our best model for the Heart Disease Prediction System. In the fig 3.2.7 we compare and choose the best model amongst given models.

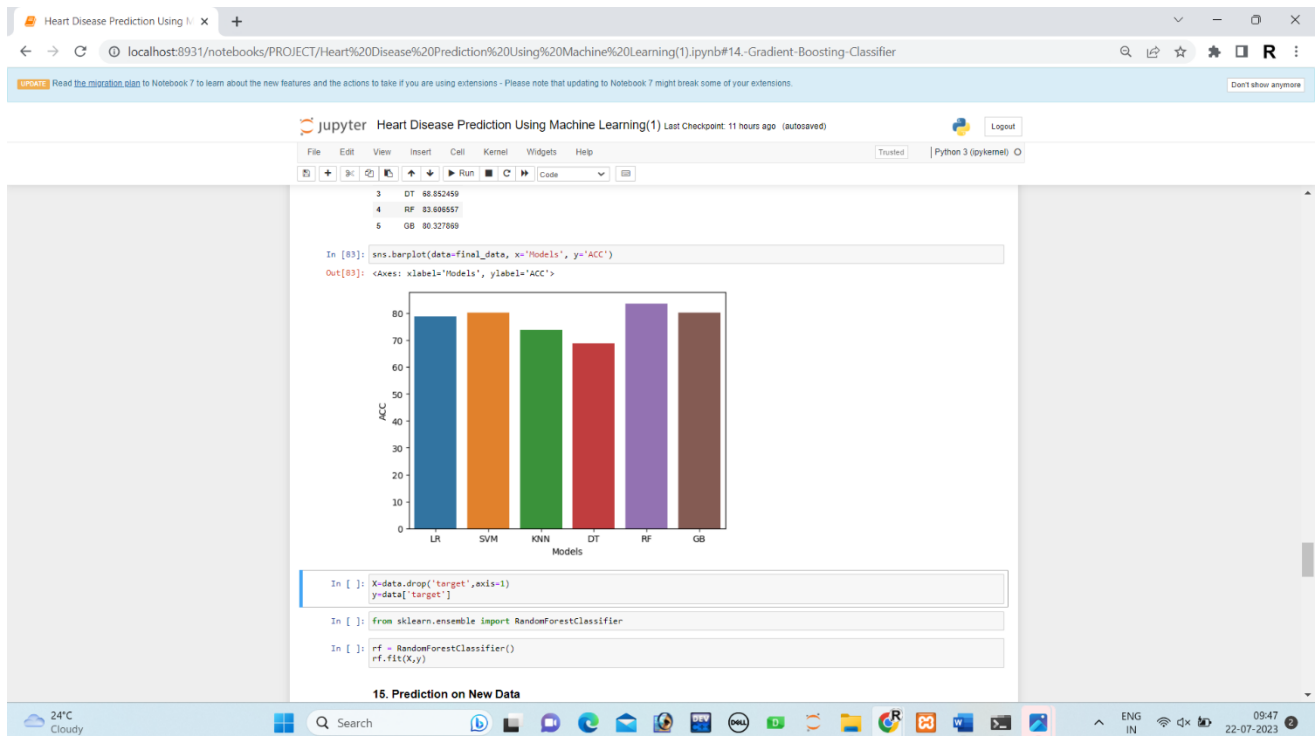
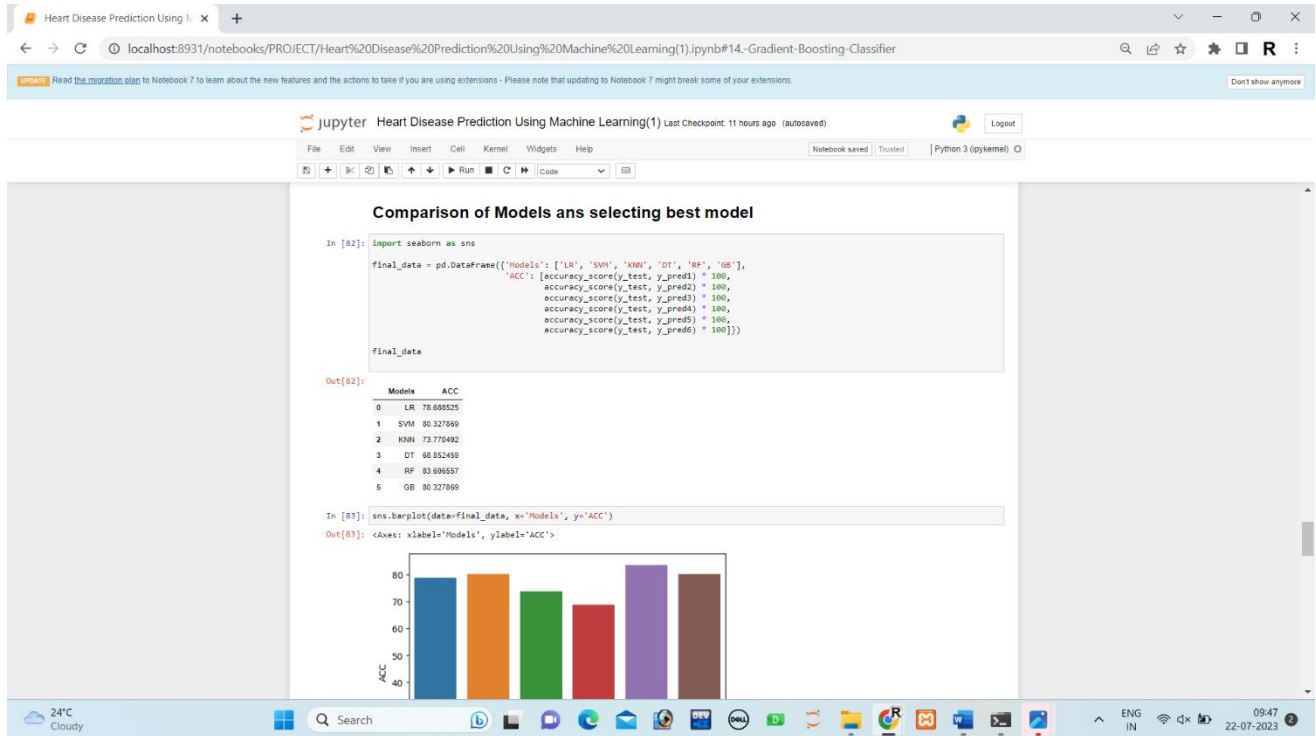
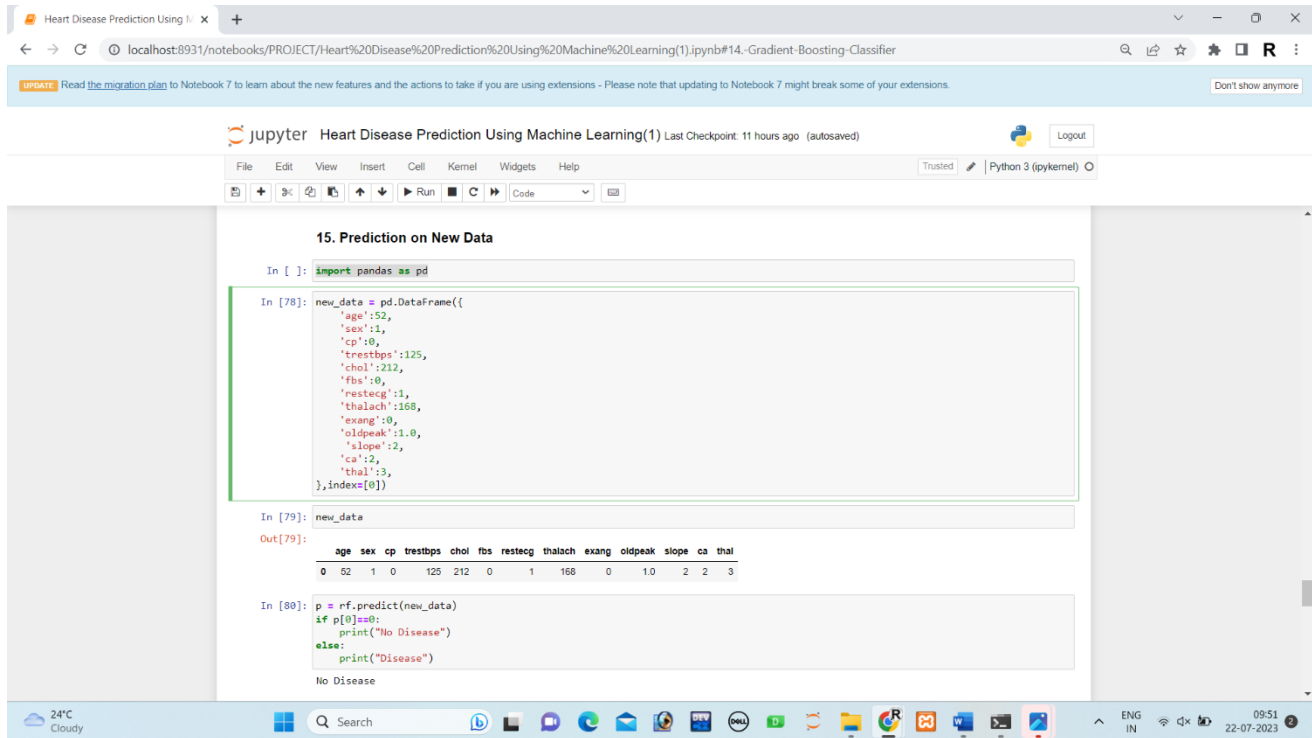


Fig:3.2.7 Choosing best model

3.2.8 Prediction on New Data

In this step, we create a new DataFrame called `new_data`, which represents the health parameters of a new individual for whom we want to predict the possibility of heart disease using Random Forest Classifier. Fig 3.2.8 snippets on prediction on new data.



The screenshot shows a Jupyter Notebook interface with the following content:

```
15. Prediction on New Data
```

```
In [ ]: import pandas as pd
```

```
In [78]: new_data = pd.DataFrame({
        'age':52,
        'sex':1,
        'cp':0,
        'trestbps':125,
        'chol':212,
        'fbs':0,
        'restecg':1,
        'thalach':168,
        'exang':0,
        'oldpeak':1.0,
        'slope':2,
        'ca':2,
        'thal':3,
        },index=[0])
```

```
In [79]: new_data
```

```
Out[79]:
```

	age	sex	cp	trestbps	chol	fbs	restecg	thalach	exang	oldpeak	slope	ca	thal
0	52	1	0	125	212	0	1	168	0	1.0	2	2	3

```
In [80]: p = rf.predict(new_data)
        if p[0]==0:
            print("No Disease")
        else:
            print("Disease")
```

No Disease

Fig:3.2.8 Prediction on new Data

3.2.9 Saving the Best Model Using Joblib

We save the best model using the Joblib library. Saving the model allows us to reuse it for future predictions without retraining.

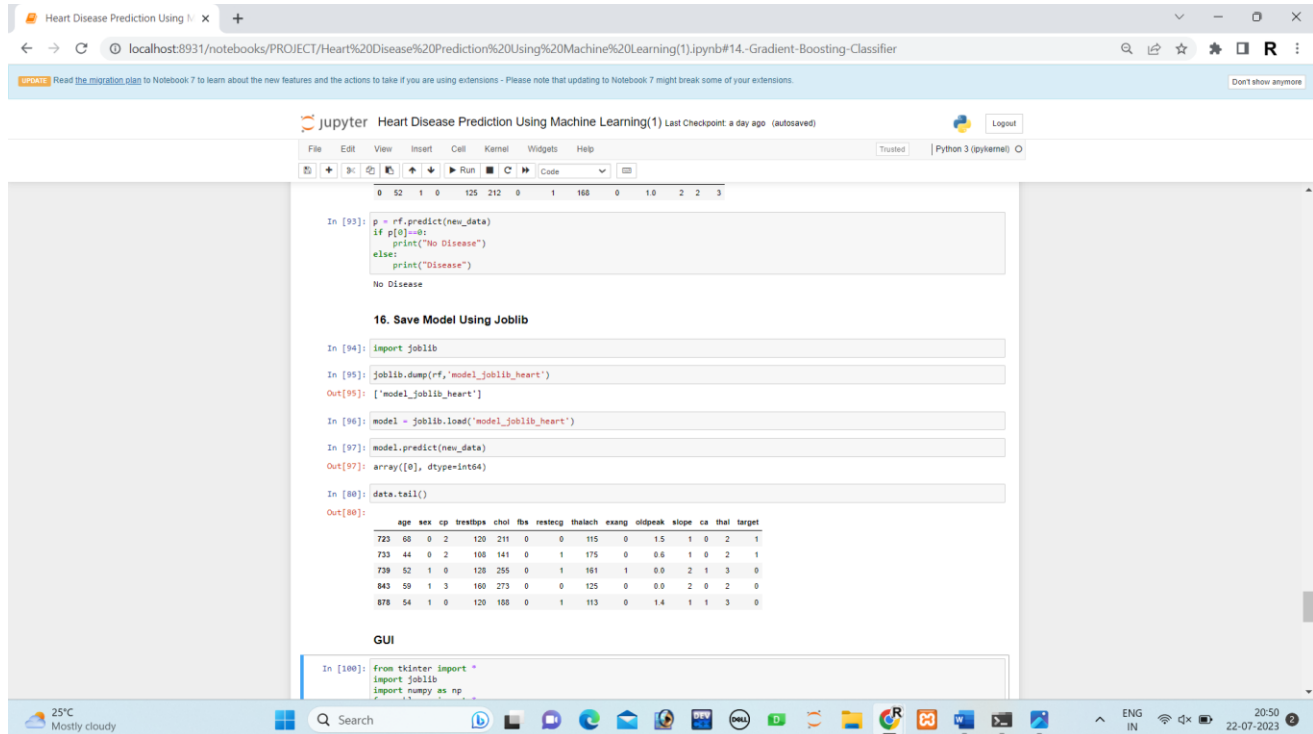


Fig:3.2.9 Saving the best model

3.2.10 GUI for Heart Disease Prediction System

The Tkinter library is employed to develop an interactive and user-friendly graphical user interface Heart Disease Prediction System. The GUI enables users to input their health data easily through various input fields and buttons. When user clicks the "Predict" button it initiates the prediction process. The Heart Disease Prediction System then uses the trained Random Forest Classifier to analyze the user's input and predict the likelihood of heart disease.

After the prediction process, the user will receive a message displayed on the GUI. If the trained model predicts a value of 0, the system will display the message "No Heart Disease," indicating that the user is not likely to have heart disease based on their input. On the other hand, if the model predicts a value of 1, the system will display the message "Possibility of Heart Disease," suggesting that the user might have an increased risk of heart disease based on their input. The fig 3.2.10 shows the output of heart disease prediction using GUI.

Heart Disease Prediction Using Machine Learning(1) Last checkpoint 13 hours ago (autosaved)

```
In [1]: from tkinter import *
import sys
import numpy as np
from sklearn import *

def show_entry_fields():
    p1 = int(e1.get())
    p2 = int(e2.get())
    p3 = int(e3.get())
    p4 = int(e4.get())
    p5 = int(e5.get())
    p6 = int(e6.get())
    p7 = int(e7.get())
    p8 = int(e8.get())
    p9 = int(e9.get())
    p10 = float(e10.get())
    p11 = int(e11.get())
    p12 = int(e12.get())
    p13 = int(e13.get())

    model = joblib.load('model.joblib')
    result = model.predict([[p1, p2, p3, p4, p5, p6, p7, p8, p9, p10, p11, p12, p13]])

    if result[0] == 0:
        result_label.config(text="No Heart Disease", fg="green")
    else:
        result_label.config(text="Possibility of Heart Disease", fg="red")

master = Tk()
master.title("Heart Disease Prediction System")

master.configure(bg="lightgray")

title_label = Label(master, text="Heart Disease Prediction System", bg="black", fg="white", font=("Helvetica", 16))
title_label.grid(row=0, columnspan=2, pady=10)

Label(master, text="Enter Your Age",).grid(row=1)
Label(master, text="Male Or Female (1/0)",).grid(row=2)
Label(master, text="Enter Value of CP",).grid(row=3)
Label(master, text="Enter Value of trestbps",).grid(row=4)
Label(master, text="Enter Value of chol",).grid(row=5)
Label(master, text="Enter Value of fbs",).grid(row=6)
Label(master, text="Enter Value of restecg",).grid(row=7)
Label(master, text="Enter Value of thalach",).grid(row=8)
Label(master, text="Enter Value of exang",).grid(row=9)
Label(master, text="Enter Value of oldpeak",).grid(row=10)
Label(master, text="Enter Value of slope",).grid(row=11)
Label(master, text="Enter Value of ca",).grid(row=12)

e1 = Entry(master)
e2 = Entry(master)
e3 = Entry(master)
e4 = Entry(master)
e5 = Entry(master)
e6 = Entry(master)
e7 = Entry(master)
e8 = Entry(master)
e9 = Entry(master)
e10 = Entry(master)
e11 = Entry(master)
e12 = Entry(master)
e13 = Entry(master)

e1.grid(row=1, column=1)
e2.grid(row=2, column=1)
e3.grid(row=3, column=1)
e4.grid(row=4, column=1)
e5.grid(row=5, column=1)
e6.grid(row=6, column=1)
e7.grid(row=7, column=1)
e8.grid(row=8, column=1)
e9.grid(row=9, column=1)
e10.grid(row=10, column=1)
e11.grid(row=11, column=1)
e12.grid(row=12, column=1)
e13.grid(row=13, column=1)

Button(master, text="Predict", command=show_entry_fields).grid(row=14, columnspan=2, pady=10)

result_label = Label(master, text="", font=("Helvetica", 12))
result_label.grid(row=15, columnspan=2)

mainloop()
```

Heart Disease Prediction Using Machine Learning(1) Last checkpoint 13 hours ago (autosaved)

```
title_label.grid(row=0, columnspan=2, pady=10)

Label(master, text="Enter Your Age",).grid(row=1)
Label(master, text="Male Or Female (1/0)",).grid(row=2)
Label(master, text="Enter Value of CP",).grid(row=3)
Label(master, text="Enter Value of trestbps",).grid(row=4)
Label(master, text="Enter Value of chol",).grid(row=5)
Label(master, text="Enter Value of fbs",).grid(row=6)
Label(master, text="Enter Value of restecg",).grid(row=7)
Label(master, text="Enter Value of thalach",).grid(row=8)
Label(master, text="Enter Value of exang",).grid(row=9)
Label(master, text="Enter Value of oldpeak",).grid(row=10)
Label(master, text="Enter Value of slope",).grid(row=11)
Label(master, text="Enter Value of ca",).grid(row=12)
Label(master, text="Enter Value of thal",).grid(row=13)

e1 = Entry(master)
e2 = Entry(master)
e3 = Entry(master)
e4 = Entry(master)
e5 = Entry(master)
e6 = Entry(master)
e7 = Entry(master)
e8 = Entry(master)
e9 = Entry(master)
e10 = Entry(master)
e11 = Entry(master)
e12 = Entry(master)
e13 = Entry(master)

e1.grid(row=1, column=1)
e2.grid(row=2, column=1)
e3.grid(row=3, column=1)
e4.grid(row=4, column=1)
e5.grid(row=5, column=1)
e6.grid(row=6, column=1)
e7.grid(row=7, column=1)
e8.grid(row=8, column=1)
e9.grid(row=9, column=1)
e10.grid(row=10, column=1)
e11.grid(row=11, column=1)
e12.grid(row=12, column=1)
e13.grid(row=13, column=1)

Button(master, text="Predict", command=show_entry_fields).grid(row=14, columnspan=2, pady=10)

result_label = Label(master, text="", font=("Helvetica", 12))
result_label.grid(row=15, columnspan=2)

mainloop()
```

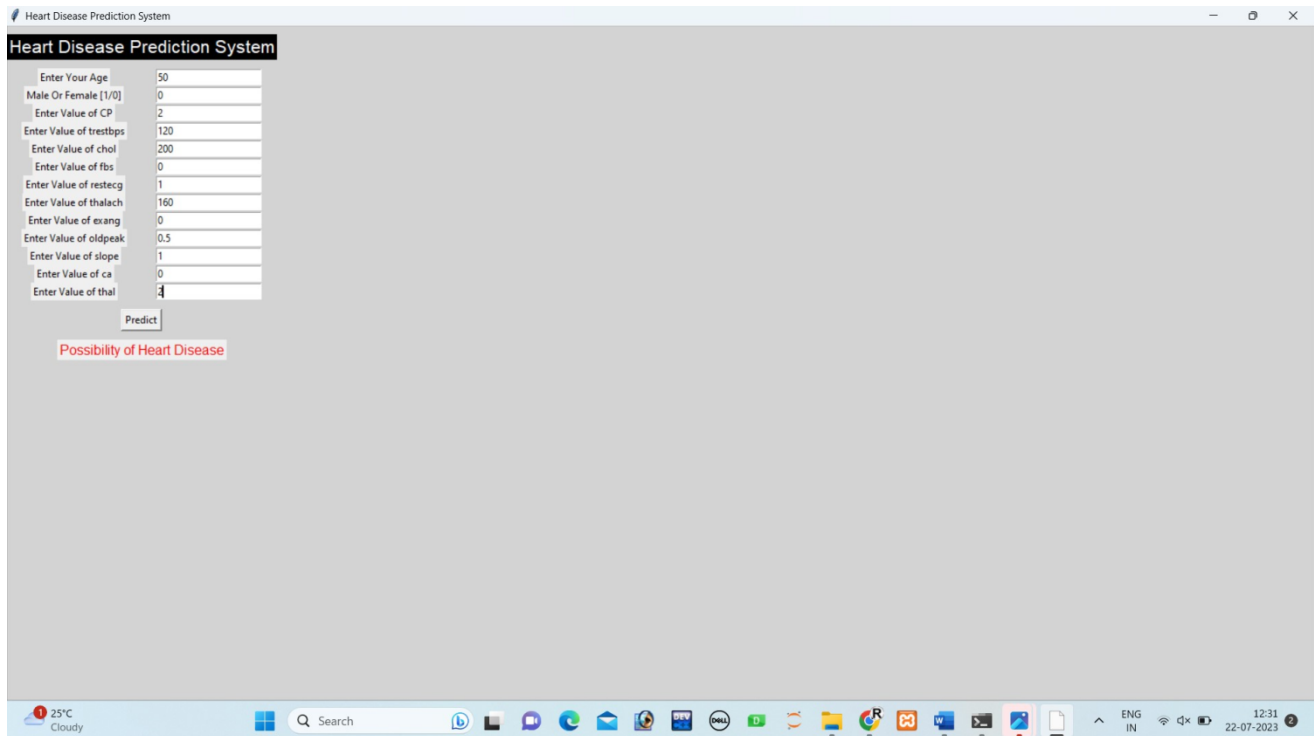
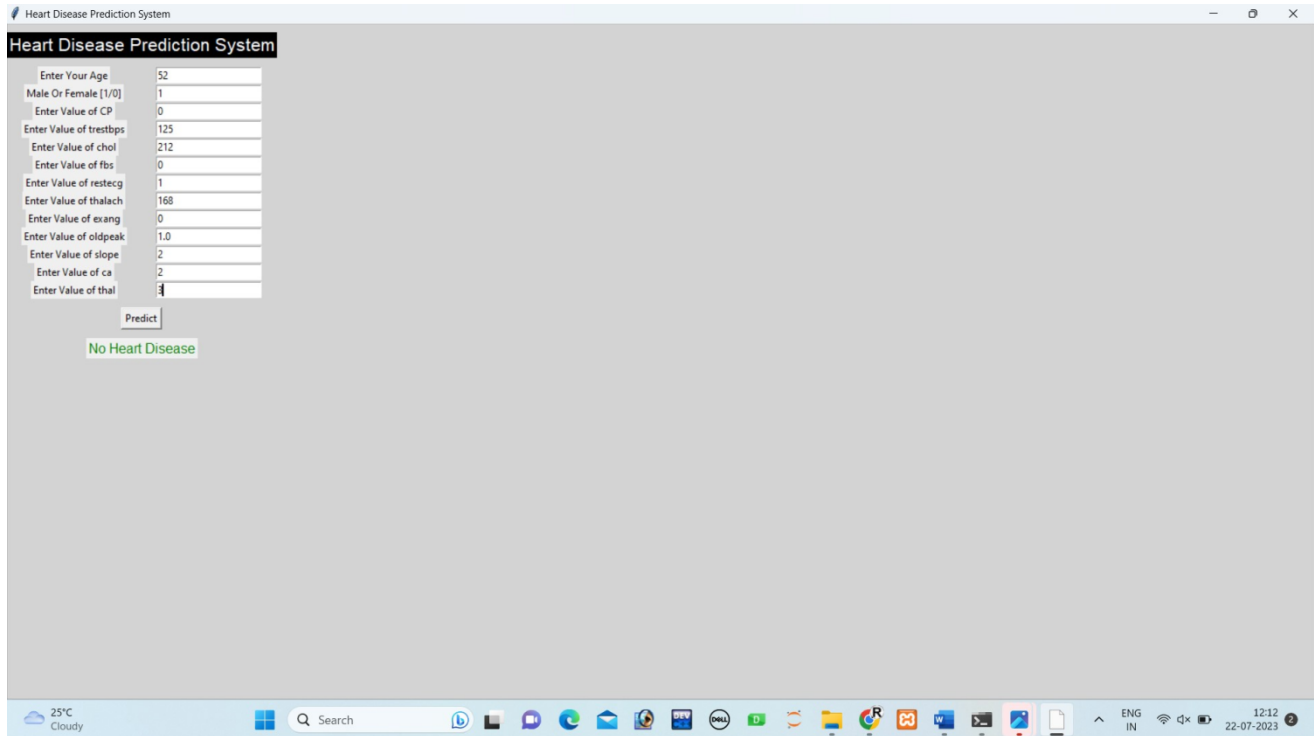


Fig:3.2.10 GUI

4.LEARNING AFTER INTERNSHIP

During my internship on AI and ML, I had the privilege of learning the foundational concepts of Python, Data Science, Artificial Intelligence, and Machine Learning. Under the guidance of experienced mentors, I delved into various projects that allowed me to apply my knowledge and gain hands-on experience.

During my internship on AI and ML, I had the opportunity to work on various projects covering a wide range of topics. I explored and gained practical knowledge in Python, Data Science, Artificial Intelligence, and Machine Learning. Through hands-on experience and guidance from mentors, I developed a deeper understanding of these fields and their applications in real-world scenarios. The projects I worked on allowed me to apply my knowledge to solve practical problems. The internship was a valuable learning experience that enriched my knowledge and expertise in AI and ML.

5.CONCLUSION

The Heart Disease Prediction System is a valuable tool that leverages machine learning algorithms to predict the likelihood of heart disease in individuals based on their health attributes. Through the implementation of various machine learning models, including Logistic Regression, SVM, KNeighbors Classifier, Decision Tree Classifier, Random Forest Classifier, and Gradient Boosting Classifier, we have successfully created a predictive system capable of assessing heart disease risk.

By working on this project, I have gained valuable knowledge and essential skills in several areas such as Machine Learning, data preprocessing, model evaluation, GUI development, and data visualization, while also contributing to a useful and impactful Heart Disease Prediction System

Overall, this project has provided me with hands-on experience in the end-to-end process of building a machine learning model for heart disease prediction, from data preprocessing to model deployment. These skills are valuable in various domains, including healthcare, finance, and more, and can serve as a solid foundation for further exploring the fascinating field of AI, data science and machine learning.

6.BIBLIOGRAPHY

Learning sites

- Google

<https://www.analyticsvidhya.com/blog/2022/02/heart-disease-prediction-using-machine-learning/>

- Dataset

<https://www.kaggle.com/datasets/johnsmith88/heart-disease-dataset>